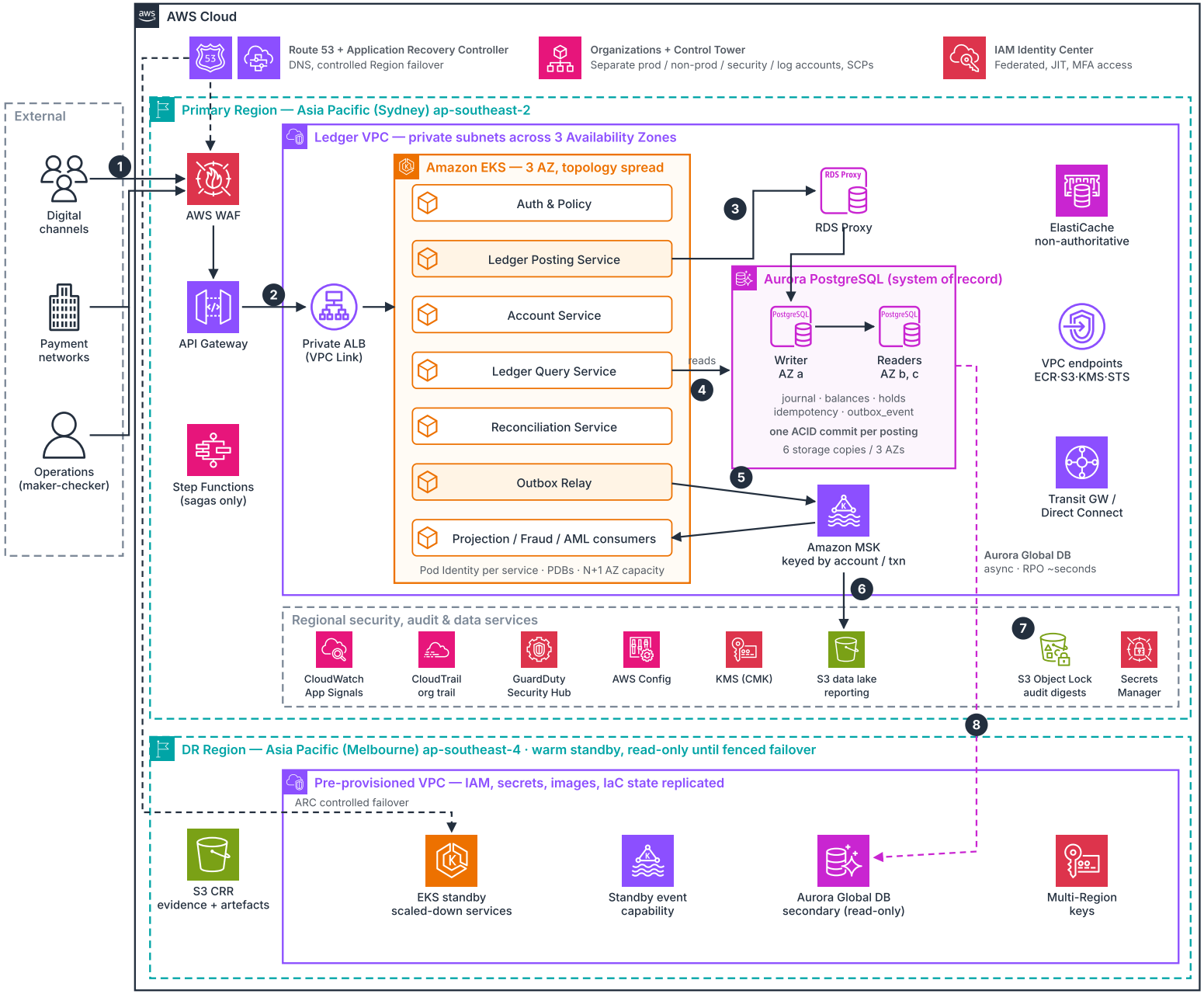
Core Ledger on AWS — Reference Architecture

Strongly consistent, append-only, double-entry posting engine on Amazon Aurora PostgreSQL · Amazon EKS · transactional outbox · Amazon MSK

REGULATED FINANCIAL LEDGER
Sydney primary · Melbourne DR



How a posting flows

1. **Ingress**
   Channels, payment networks and operators resolve through **Route 53**; **AWS WAF** and **API Gateway** authenticate, throttle, validate schema and stamp a correlation ID.

2. **Private hop**
   API Gateway reaches a **private ALB** via VPC Link. All EKS nodes and data stores run in private subnets only.

3. **Atomic posting**
   The **Posting Service** opens one Aurora transaction through **RDS Proxy**: idempotency insert, deterministic row locks, balanced debit/credit lines, balance buckets and outbox row — **one commit**.

4. **Strong reads**
   The **Query Service** serves statements and history from **Aurora readers**; writer is never loaded by read traffic.

5. **Outbox relay**
   After commit, the **Outbox Relay** publishes to **Amazon MSK**, partitioned by account or transaction ID. Delivery is at-least-once.

6. **Projections**
   Idempotent consumers dedupe by `event_id` and build balance views, fraud/AML features and the **S3 data lake**. All are rebuildable.

7. **Audit & keys**
   Customer-managed **KMS** keys everywhere; CloudTrail, Config, GuardDuty and daily signed ledger digests land in **S3 Object Lock**.

8. **Disaster recovery**
   Aurora Global Database replicates to Melbourne. Failover is a fenced, runbook-driven **ARC** switch — never active-active posting.

System of record: Aurora only. Kafka, cache, search and reporting are derived projections that can be rebuilt from committed ledger data.

Core Ledger on AWS — Design Controls & Implementation

Principles, data model, security, resilience and operating targets that make the architecture auditable

EXACTLY-ONCE FINANCIAL EFFECT
AT-LEAST-ONCE DELIVERY

Design principles

1. **The DB transaction is the accounting boundary** — transaction, all legs, balances, idempotency record and outbox event commit atomically.
2. **Double entry is enforced** — debits = credits per ledger, currency and value date, or the commit fails.
3. **Posted history is never edited** — corrections are linked reversals; app roles have no `UPDATE` / `DELETE` on journals.
4. **One authoritative writer** — DR stays read-only until the old writer is fenced.
5. **Events are consequences of commits** — transactional outbox, no dual writes.
6. **Idempotent everywhere** — unique keys guarantee one posting; consumers dedupe by event ID.
7. **Reads never endanger posting** — replicas and projections serve reporting.
8. **Reconciliation is a first-class service** — journal, balances, events and settlement continuously proven.

Posting transaction (critical path)

1. Validate schema, currency precision, source authorisation, size.
2. Begin tx at `READ COMMITTED` + explicit locks (`SERIALIZABLE` only if load-tested).
3. Insert idempotency key — replay returns stored result; differing payload is rejected.
4. Lock balance rows in deterministic account-ID order.
5. Apply approved posting-rule version; check status, limits, holds, overdraft.
6. Write header, entry, debit/credit lines; constraints verify balance.
7. Update balance buckets; insert outbox rows with deterministic IDs.
8. Commit once; return immutable transaction ID.

Audit evidence

- Immutable journal + reversal history; PostgreSQL audit logs.
- Org CloudTrail to log-archive account; EKS audit logs; signed deployment provenance.
- Daily control totals with Merkle-root digests, KMS-signed, in S3 Object Lock.

Ledger domain model

Entity	Key rule
`ledger`	Legal accounting boundary; allowed currencies
`account`	Immutable ID; versioned state; controlled currency
`financial_transaction`	Unique ID + idempotency key; reversal link
`journal_entry`	Immutable sequence; posting-rule version
`journal_line`	Positive minor-unit amount; direction enum
`account_balance`	Updated in same tx; monotonic version
`balance_bucket`	Available, posted, pending, held, credit limit
`hold`	Create/adjust/capture/release/expire — idempotent
`idempotency_record`	Unique tenant+source+key; request hash
`outbox_event`	Event ID, aggregate sequence, schema version
`posting_rule`	Versioned; approved versions immutable
`reconciliation_result`	Control ID, totals, variances, evidence

Database invariants

- Σ debits = Σ credits per ledger, currency and entry; ≥ 2 lines per entry.
- Money as integer minor units or constrained `NUMERIC` — **never floating point**; FX uses explicit position and gain/loss legs.
- Reversals reference one original and cannot exceed the unreversed amount.
- Backdated postings keep a controlled value date and the real system timestamp.
- Partition journals by posting month; size partitions only after load tests.

Domain events (MSK)

`LedgerTransactionPosted.v1` `LedgerTransactionReversed.v1` `AccountBalanceChanged.v1` `HoldCreated/Captured/Released.v1` `ReconciliationExceptionRaised.v1` — envelope carries `event_id`, `aggregate_sequence`, `committed_at`, `schema_version`, `trace_id`.

Resilience & DR targets

30 s	15 min	99.99%
Provisional RPO	Provisional RTO	Posting API availability / month

- 3-AZ EKS, ≥ 3 replicas per critical service, PDBs, capacity to lose one AZ; test with EKS zonal shift.
- Aurora writer + ≥ 2 readers in separate AZs; RDS Proxy, bounded pools, lock/statement timeouts.
- Failover runbook: freeze posting → fence old writer → promote secondary → validate control totals → resume → record recovery point.
- Continuous backups (≤ 35-day PITR), cross-account snapshot copies, automated restore tests with reconciliation.

Targets require business-impact-analysis approval and test evidence; they are not AWS guarantees.

Security controls

- IAM Identity Center federation; no routine IAM users or long-lived keys.
- EKS Pod Identity / IRSA — least-privilege role per service.
- Separate app, migration, read-only, reconciliation and break-glass DB roles.
- Maker-checker for posting rules, manual adjustments, failover and policy changes.
- Customer-managed KMS keys per environment and data class; TLS everywhere; AWS Payment Cryptography if card/PIN is in scope.
- Egress via network firewall, DNS controls and allow-lists; VPC endpoints for AWS APIs.

Do not use Amazon QLDB. AWS ended support on 31 July 2025. Aurora's append-only design does not give cryptographic proof by itself — implement and independently review the digest scheme above.

Source: AWS Core Ledger Architecture and Implementation Plan · Sydney/Melbourne feature parity to be confirmed in detailed design